Managing Azure Resources with Terraform SOP

Cognizant

Table of Contents

1. Managing Azure Resources with Terraform 3
1.1 Description 3
1.2 Architecture Diagram 3
1.3 Lab Steps 4
1.4 Troubleshooting 6
1.5 Supporting References 6

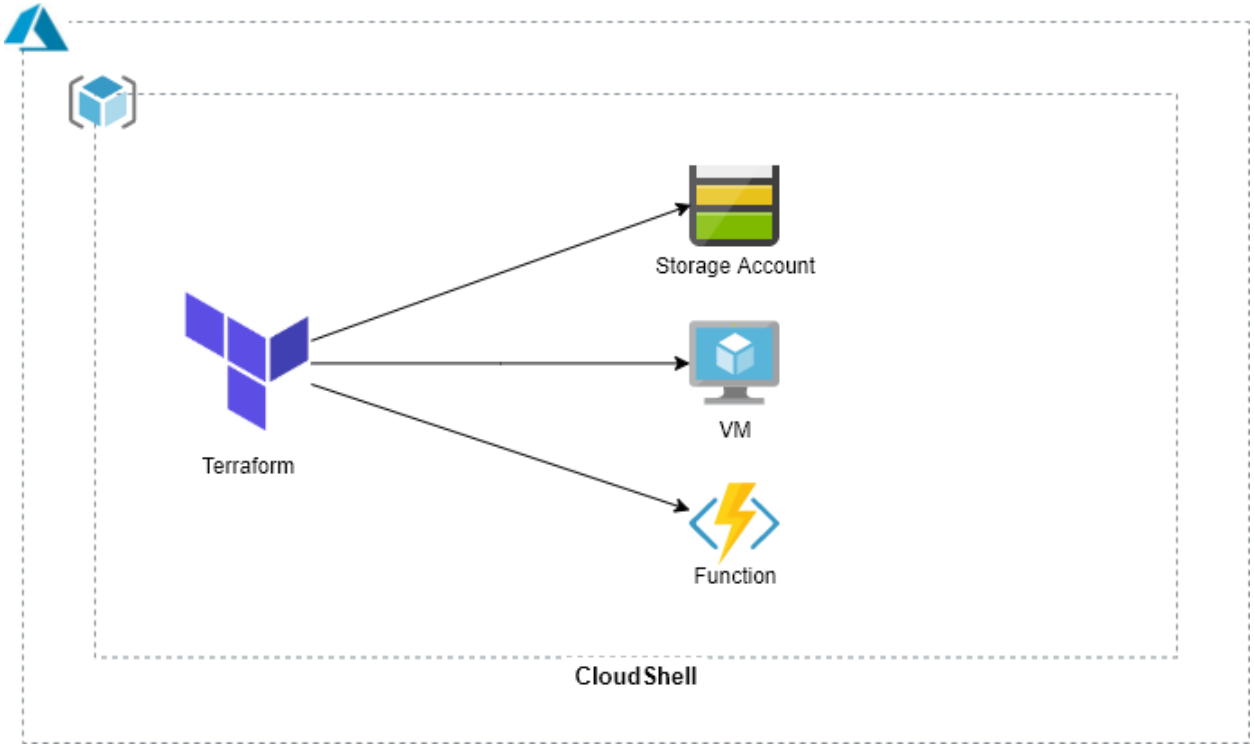
1. Managing Azure Resources with Terraform

1.1 Description

HashiCorp Terraform is an open-source tool for provisioning and managing cloud infrastructure. It codifies infrastructure in configuration files that describe the topology of cloud resources. The Terraform CLI provides a simple mechanism to deploy and version the configuration files to Azure
In this lab exercise, you will be creating an azure windows virtual machine using terraform which is open-source infrastructure as code software tool. Using terraform azure provider you can configure infrastructure in Microsoft Azure.

1.2 Architecture Diagram

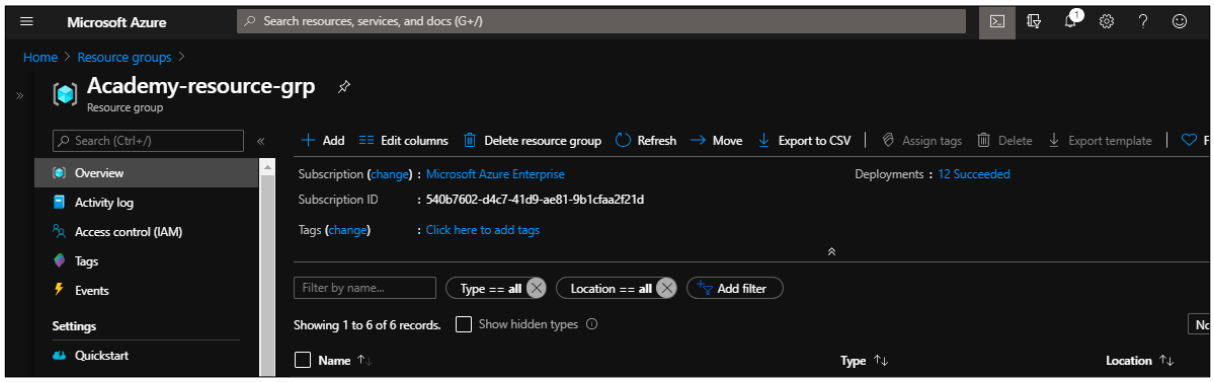
The diagram below displays a visual representation of the application architecture:



1.3 Lab Steps

Follow the steps outlined below to achieve the objective of this lab exercise:

1. Log in to the Azure Portal and go to your respective resourcegroup with your empid.
2. Terraform installation in CloudShell:
   a. Inside RG, open CloudShell by clicking on CloudShell icon.



   b. CloudShell will use a storage account internally, if it requires you to create one, create or else continue.
   c. Navigate to cloud drive directory which is available by default:

```
cd clouddrive
```

   d. Create a folder named terraform where all the required files can be placed.

```
mkdir terraform
```

   e. Start with installation of Terraform and unzipping the binaries.
   f. Install the binary from: https://www.terraform.io/downloads.html and choose linux binary and download using wget.
   g. Unzip the binary.
   h. Verify that, terraform is installed properly using the following command:

```
terraform -version
```

3. Terraform code:
   a. Now let us try to understand azure terraform provider. The Azure Provider can be used to configure infrastructure in Microsoft Azure using the Azure Resource Manager APIs.

```
# Configure the Azure Provider
provider "azurerm" {
  # whilst the `version` attribute is optional, we recommend
pinning to a given version of the Provider
  version = "=2.20.0"
  features {}
}
```

b. Build a windows VM using Terraform.
Note: You can find all the resources examples here:
https://www.terraform.io/docs/providers/azurerm/index.html

c. Take a look at virtual machine block code:

```
resource "azurerm_windows_virtual_machine" "example" {
  name                = "mywinvm"
  resource_group_name = "${var.rgname}"
  location            = "centralus"
  size                = "Standard_B1s"
  admin_username      = "adminuser"
  admin_password      = "P@$$w0rd1234!"
  network_interface_ids = [
    azurerm_network_interface.example.id,
  ]

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2016-Datacenter"
    version   = "latest"
  }
}
```

From the above code you can find that you are specifying the azure vm properties such as vm name, size, region, admin username and password etc.,

d. Since a vm requires virtual network, subnet and network interface, add the following complete code into a main.tf file in CloudShell.

e. Create a file named **main.tf** and copy the following code:

f. Now that main part of the code is in place, we need to understand how to declare runtime variables. In this case, resource group name should be declared at run time.

g. We should perform that using variables. From the code if you notice resource group name is dynamic:

```
resource_group_name = "${var.rgname}"
```

h. Create another file named **var.tf** in CloudShell and add the following code:

```
variable "rgname" {
  type        = "string"
  description = "Enter the Resource Group Name."
}
```

i. Initialize a working directory containing Terraform configuration files:

```
terraform init
```

j. Run the following command to apply changes required to reach the desired state of the configuration. This will start creating the azure virtual machine:

```
terraform apply -auto-approve
```

k. The above command will ask to enter the resource group name. Enter the rg name which you are in. This will start creating the vm.

```
azureuser@Azure:~/clouddrive/terraform$ terraform apply -auto-approve
var.rgname
  Enter the Resource Group Name.

  Enter a value: Academy-764854-RG

var.rgname
  Enter the Resource Group Name.

  Enter a value: Academy-764854-RG

azurerm_virtual_network.example: Refreshing state... [id=/subscriptions/540b7602-d4c7-41d9-ae81-9b1cfaa2f21d/resourceGroups/Academy-764854-RG/providers/Microsoft.Network/virtualNetworks/test-n
etwork]
azurerm_subnet.example: Refreshing state... [id=/subscriptions/540b7602-d4c7-41d9-ae81-9b1cfaa2f21d/resourceGroups/Academy-764854-RG/providers/Microsoft.Network/virtualNetworks/test-network/su
bnets/internal]
azurerm_network_interface.example: Refreshing state... [id=/subscriptions/540b7602-d4c7-41d9-ae81-9b1cfaa2f21d/resourceGroups/Academy-764854-RG/providers/Microsoft.Network/networkInterfaces/ex
ample-nic]
```

l. Run the following command to display the human-readable output from a state or plan file:

```
terraform show
```

m. Check your resourcegroup as your virtual machine is successfully created.

1.4 Troubleshooting

S. No	Problem	Solution
1	Error when terraform apply command is given	• Make sure that you have selected the correct resource group name. • Make sure that you have the latest version of Terraform.
2	Terraform apply fails with lock	Make sure that you are in cloud drive folder.

1.5 Supporting References

Refer the below links for additional information:

1. https://www.terraform.io/docs/providers/azurerm/r/virtual_machine.html
2. https://github.com/Azure/terraform/tree/master/quickstart